

Board Forge: A Self-Improving PCB Design Harness

Team brief · MongoDB Harness Engineering & Model Wrangling Hackathon · Sat Sept 26, 2026 · Board Forge is a working name

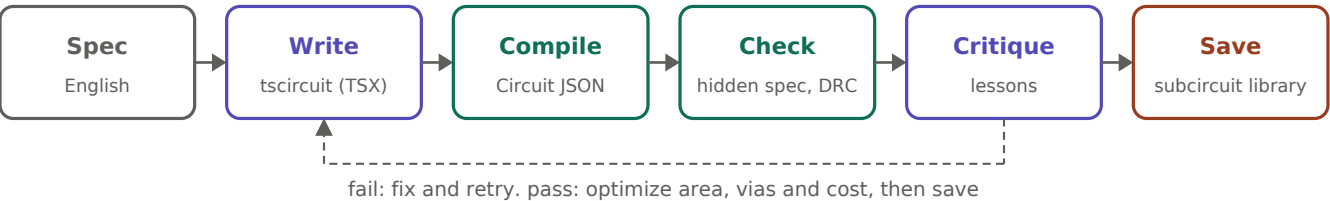
The pitch

Board Forge is a harness that designs circuit boards from an English spec, checks them against a hidden spec it never sees, and rewrites its own rules, context and workflow based on what fails. Boards are written in tscircuit, an open-source TypeScript and React framework, so the whole team can read and write the core code. PCB design is the proving ground; the harness is the product.

"The model didn't get smarter. The harness did."

- **Statement One (Recursive Harnessing):** the harness keeps its rules, context policy, guardrails, tool access, workflow and model routing in a versioned config, and a meta-agent rewrites that config when the metrics say it should.
- **Statement Two (Long Horizon):** a large board is split into a queue of subcircuits, built and verified step by step, checkpointed throughout, and resumable after a crash. Context never grows; every call is rebuilt from Atlas.

The design loop



1. **Spec in:** e.g. "a board that reads a temperature sensor over I2C, powered from USB-C, with a status LED".
2. **Write:** the coder agent writes tscircuit components, reusing verified subcircuits from the library.
3. **Compile:** tscircuit turns the code into Circuit JSON.
4. **Check** against the hidden spec: connectivity, design rules in code, autorouting with zero DRC errors, SPICE for analog parts.
5. **Measure:** board area, via count, trace length, parts cost from the BOM.
6. **Critique:** failures go to the critic; new patterns become lessons.
7. **Save:** verified subcircuits go into the library.

The harness: a versioned config that rewrites itself

The harness's behavior lives in a config document in Atlas, not in code. Each generation is a new version. A meta-agent reads failures across many runs, proposes config changes, and keeps a change only if the next batch of boards scores better.

Config controls	Example change the meta-agent might make
Rules and guardrails	Add "I2C lines need pull-up resistors" to the coder's rules
Context policy	Retrieve 2 subcircuits instead of 5 because context overload is causing errors
Tool access	Block autorouting until connectivity checks pass; narrow the parts whitelist
Workflow	Plan subcircuits before writing; place sensors before routing; split boards over N parts

Config controls	Example change the meta-agent might make
Repair budget	Allow 3 repair attempts before escalating to the planner
Model routing	Cheap model writes code; strong model critiques and plans

A guardrail on the thing that rewrites guardrails: every proposed config change is tested before it is kept. A change that lowers the score is rolled back automatically, and a change that would weaken or bypass a hidden check is rejected. Every proposal and verdict is stored so judges can see the harness refuse bad changes.

Harness capabilities to make visible

- **Memory in three layers:** working memory (the current board's state, rebuilt from Atlas each call, so context never grows); episodic memory (every run and failure, searchable by vector similarity); semantic memory (distilled lessons and verified subcircuits). Show a retrieved lesson changing the agent's behavior.
- **Guardrails:** the agent can never see or edit the hidden checks; parts and footprints come from a whitelist; checks gate each step; each board has a cost cap.
- **Long horizon:** the finale is a bigger board (USB-C power, regulator, microcontroller, two sensors, LEDs). The harness splits it into subcircuits in a work queue in Atlas, builds and verifies each, checkpoints after every step, then assembles them. Kill it partway through; it resumes from the queue with nothing lost.
- **Model wrangling:** a router logs cost and outcome per role, so the harness can show which model is worth paying for where.
- **Observability:** every call, tool result and config version is stored, so any board's history can be replayed, and a "why?" view traces a design choice back to the lesson or rule that caused it.

Proving the harness matters: the ablation

Run the same model on the same held-out specs under different harness versions. This table answers "is the harness actually doing anything?" and is the most persuasive single artifact in the demo.

Setup	Hidden checks passed	Attempts per board	Cost per board
Bare model, no harness	measured	measured	measured
Harness v0 (loop + checks)	measured	measured	measured
Harness vN (evolved config, library, lessons)	measured	measured	measured
Optional: cheap model + vN vs. frontier model, no harness	measured	measured	measured

The last row also covers the "model wrangling" half of the event: a cheap model inside a strong harness beating a frontier model on its own.

Checks and metrics

Check	What it proves	How
Connectivity	Every required connection exists (SDA to MCU pin, pull-ups present, grounds tied)	Compare Circuit JSON nets with the hidden spec
Design rules	Common first-board mistakes are absent	Geometric checks in code, e.g. a decoupling capacitor within a few mm of every chip's power pin
Routing and DRC	The board is manufacturable as laid out	tscircuit autorouter plus its design rule checks: zero errors
SPICE	Analog values are right (dividers, filters)	tscircuit SPICE simulation against expected values
Quality	The board improves over generations	Area, vias, trace length, BOM cost

MongoDB Atlas

Collection	One document is	Used for
harness_versions	a harness config	Rules, context policy, tool access, workflow, routing, parent version, scores, the meta-agent's rationale and verdict
specs	a board task	Spec text, pointer to its hidden checks, train or held-out label
subcircuits	a verified building block	Code, description embedding, checks passed, reuse count
boards	a board version	Code, Circuit JSON, metrics, harness version used, subcircuits used
work_queue	a step of a large board	Status, dependencies, checkpoint; drives kill-and-resume
runs	one call or tool result	Structured results, cost, model used; the replayable trace
lessons	a learned design rule	Pattern, fix, embedding, how often it helped

- **Vector Search** retrieves subcircuits, similar past failures and relevant lessons.
- **Versioned documents** make harness evolution, board history and rollback simple queries.
- **Change streams** trigger the critic when a run finishes and the meta-agent when a batch completes.
- **Transactions and idempotent writes** make each work-queue step atomic, so a crash never leaves half a step.

Demo (3 minutes, live, no slides)

Time	On screen	Point made
0:00-0:40	A judge's spec; the first attempt fails a hidden check; the lesson and fix appear	It checks itself against specs it can't see
0:40-1:20	The config diff: what the harness changed about itself, and a change it rejected	The harness evolves, with guardrails
1:20-2:00	The ablation table: same model, bare vs. v0 vs. evolved	The harness is doing the work
2:00-2:40	The big board building from its queue; kill the worker, restart, it resumes	Long horizon and durability
2:40-3:00	"The model didn't get smarter. The harness did."	The thesis

Prerecord a full run as a fallback, and cut a 1-minute version for the submission video.

Priorities if time runs short

1. **Must have:** the loop with hidden checks, the subcircuit library, lessons retrieved from memory, and a versioned harness config.
2. **High value:** the meta-agent evolving the config, and the ablation table.
3. **Strong bonus:** the long-horizon finale board with kill-and-resume.
4. **Nice to have:** the cheap-vs-frontier ablation row and the "why?" trace view.

Roles

Who	Owns
Marcos	Specs and hidden checks (connectivity and design rules), electronics judgment, critic prompt, lesson quality
Teammate 1	Harness runtime: config versioning, meta-agent, rollback and rejection logic, work queue, kill-and-resume
Teammate 2	tscircuit coder agent; wiring compile, autoroute and DRC into structured results; model router; ablation runs

Who	Owns
Teammate 3	UI (PCB and schematic views, config diff viewer, ablation table, "why?" view), demo script, video

Timeline (10:30 AM - 5:00 PM)

Time	Goal
Before 10:30	One teammate installs tscircuit on every laptop and builds an LED-plus-resistor board through the autorouter
10:30-11:00	Freeze the harness config, result JSON and Atlas schemas; pin the tscircuit version; repo public
11:00-12:30	Marcos: first specs and hidden checks. Team: loop, Atlas, config versioning, UI on sample boards
12:30	Checkpoint: one board goes spec to routed PCB, passes hidden checks, under a stored config version
12:30-2:00	Critic and lessons, subcircuit reuse, more specs; split specs into train and held-out
2:00-3:00	Meta-agent evolving the config, with rollback and rejection; work queue and kill-and-resume
3:00-3:30	Run the ablation on held-out specs; feature freeze at 3:30; record a full backup run
3:30-4:30	Polish and rehearse twice
4:30-5:00	Record the 1-minute video, submit, confirm the repo is public

Risks

Risk	Mitigation
Evolved config doesn't beat v0	Show the loop with its rejected and rolled-back changes honestly; the ablation still shows v0 beating the bare model
Ablation runs take too long	Small held-out set (4-6 specs), parallel runs, cached compiles
tscircuit version changes break things	Pin one version at 10:30 and don't upgrade
Autorouter slow or fails on a board	Keep boards small (2 layers, under ~20 parts); cache routed results
Agent picks parts or footprints that don't exist	Restrict it to a whitelist of known-good parts and footprints
Clean checks but a board that wouldn't work	State the limit honestly: checks prove spec and rule compliance, not a working board

Rules to respect: build in the provided Atlas Sandbox, keep the repo public, write all code and specs during the event, and have someone free for MongoDB.local on Wed Sept 30 from 10 AM if you make the finals.